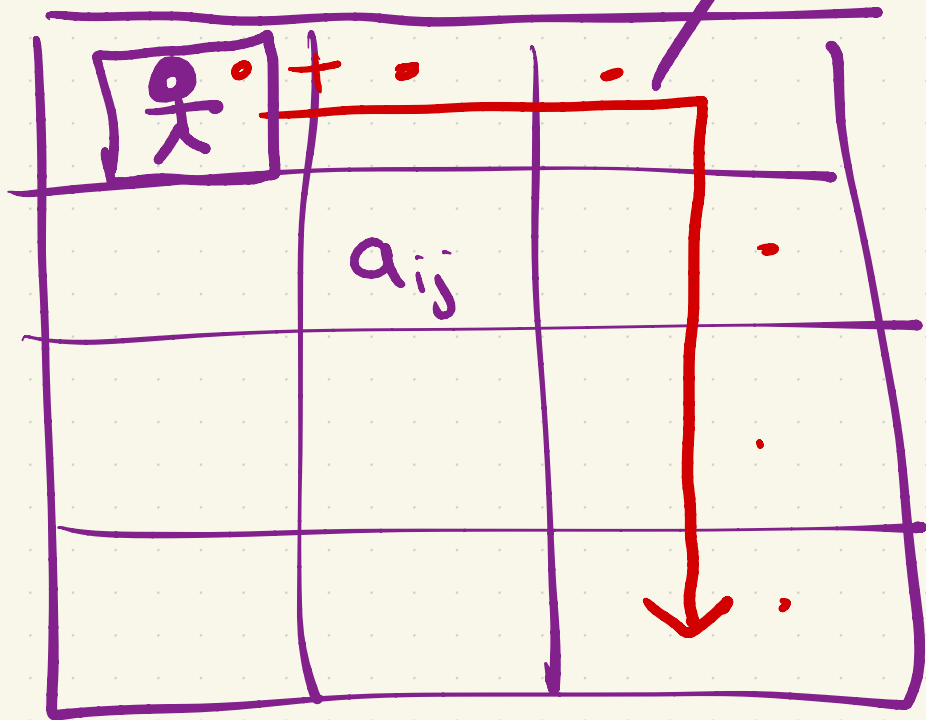




Answer Construction & Non Integer DP States

- Gaurish Baliga

$n \times m$



Right
Down

→ min possible
cost

$n \times m$ grid

1



4

1



1



1

2

5

3

2



8

→ right
↓ down

(i, j) $\xrightarrow{\text{Right}} (i, j+1)$

$\downarrow \text{down}$
 $(i+1, j)$

$dp[i][j] =$
min cost to get
from cell (i, j) to
 $(n-1, m-1)$

$$dp[i][j] = a_{ij} + \min($$

minimum
?

→

$dp[i][j+1],$

↓

$dp[i+1][j])$

2nd Task

#	2nd	Task
1	2	3
4	8	2
1	1	1

Red arrows indicate a sequence: from (1,2) to (4,8) to (1,1), and from (1,1) to (2,8) and (3,2).

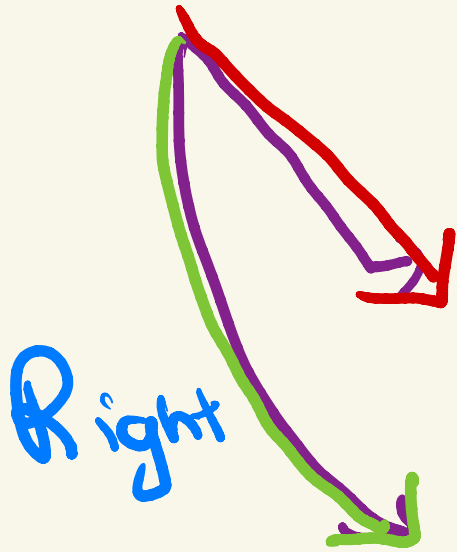
min cost

8	
0	0
1	0
2	0
2	1
2	2

Blue arrows indicate a sequence: from (0,0) to (1,0) to (2,0) to (2,1) to (2,2).

$$dp_{ij} = a_{ij} + \min \left(\overset{\text{Down}}{\underbrace{dp_{i+1,j}}}, \right.$$

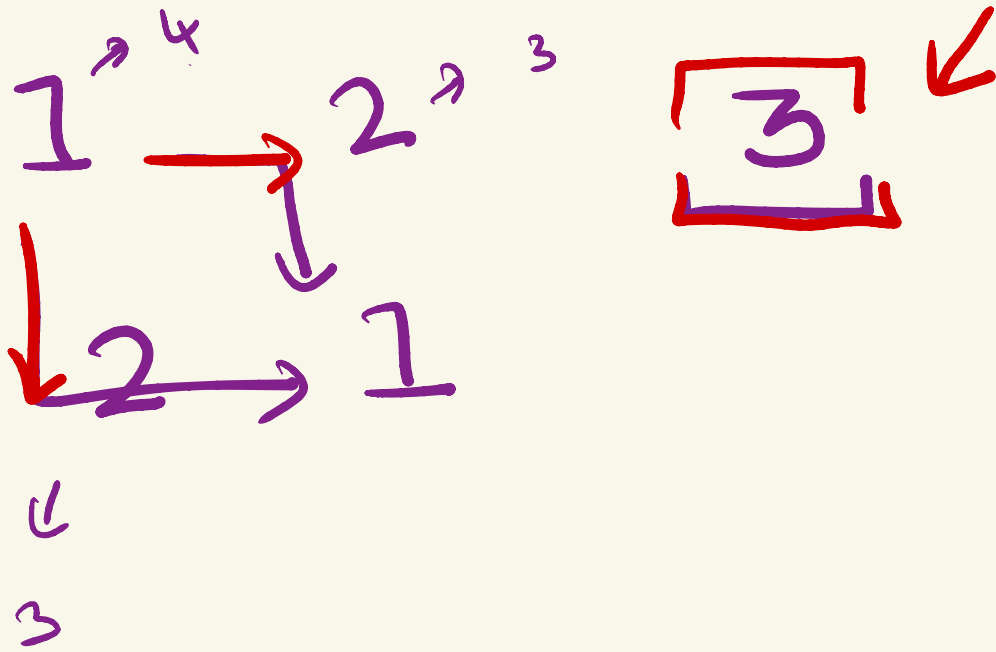
$$\left. \underbrace{dp_{i,j+1}}_{\text{Right}} \right)$$



Down

$$a_{ij} + dp_{i+1,j}$$

$$a_{ij} + dp_{i,j+1}$$



2 transitions

100 transitions

[100 nested if else !?

dp is $\rightarrow$ pair <int, int>.

↑ ↑

answer choice

x

0 1 2 ... $x-1$

└────────────────────────────────┘



Answer Construction

- Instead of just finding an optimal value among multiple possibilities, we also want to find the exact possibility that generates that value.
 - Ex: Find the actual path with the minimum sum in the Grid problem
- At every state we make some optimal choice.
 - If we store the best choice for every state, we won't have to try them all out while constructing the answer.
- Start from the state that contains your final subproblem and keep making the best choice (which was already stored) until you reach the end.

Answer Construction - Grid Problem



```
int n = 3, m = 3;
vector<vector<int>> grid(3, vector<int>(3));
vector<vector<pair<int, int>>> dp(n, vector<pair<int, int>>(m, {-1, 0}));
// 0 -> take a down direction
// 1 -> take a right direction
int f(int i, int j){
    if(i == n || j == m)
        return 1e9;
    if(i == n - 1 && j == m - 1)
        return grid[n - 1][m - 1];
    if(dp[i][j].first != -1)
        return dp[i][j].first;

    int ans1 = f(i + 1, j);
    int ans2 = f(i, j + 1);

    if(ans1 < ans2){
        dp[i][j].second = 0;
    }else{
        dp[i][j].second = 1;
    }

    return dp[i][j].first = grid[i][j] + min(ans1, ans2);
}
```



down
right

same

recursive
dp
solution

Answer Construction - Grid Problem



```
void solve(){
    for(int i = 0; i < 3; i++){
        for(int j = 0; j < 3; j++){
            cin >> grid[i][j];
        }
    }
    cout << f(0, 0) << nline;
    → pair<int, int> current = {0, 0};
    while(current != mp(n - 1, m - 1)){
        cout << current.first << " " << current.second << nline;
        if(dp[current.first][current.second].second == 0)
            current.first++;
        else
            current.second++;
    }
    cout << current.first << " " << current.second << nline;
}
```

Handwritten annotations:

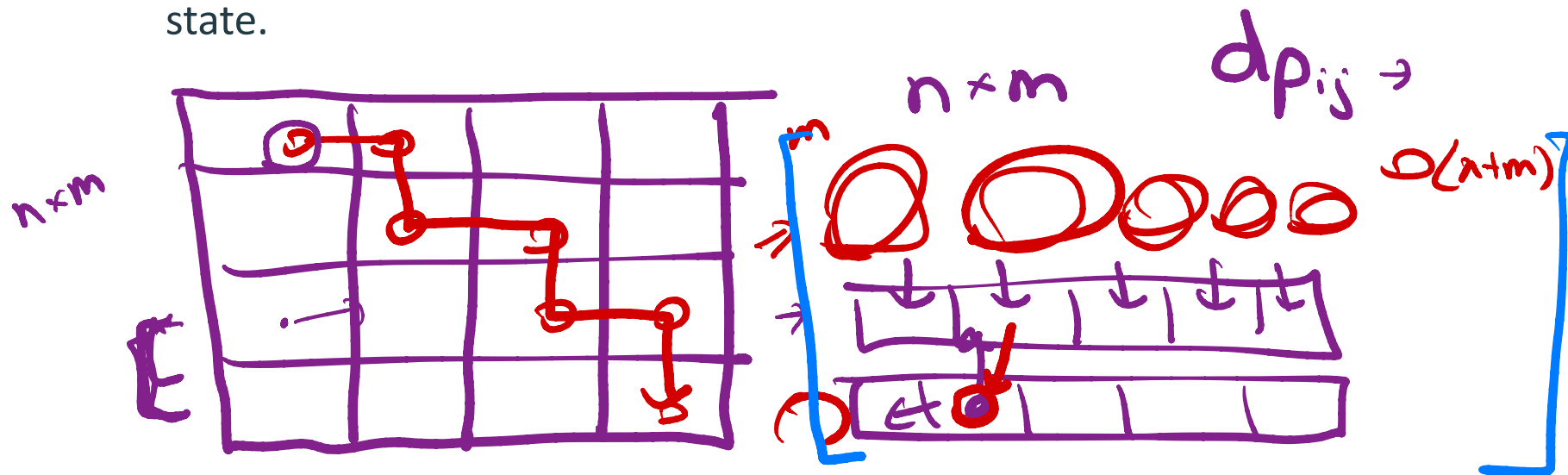
- Backtracking** (blue bracket on the left side of the while loop)
- same** (blue arrow pointing to the initial current value)
- make-pair(n-1, m-1)** (purple text with an arrow pointing to the target value in the while loop condition)
- Down** (blue text next to the first++ increment)
- Right** (blue text next to the second++ increment)

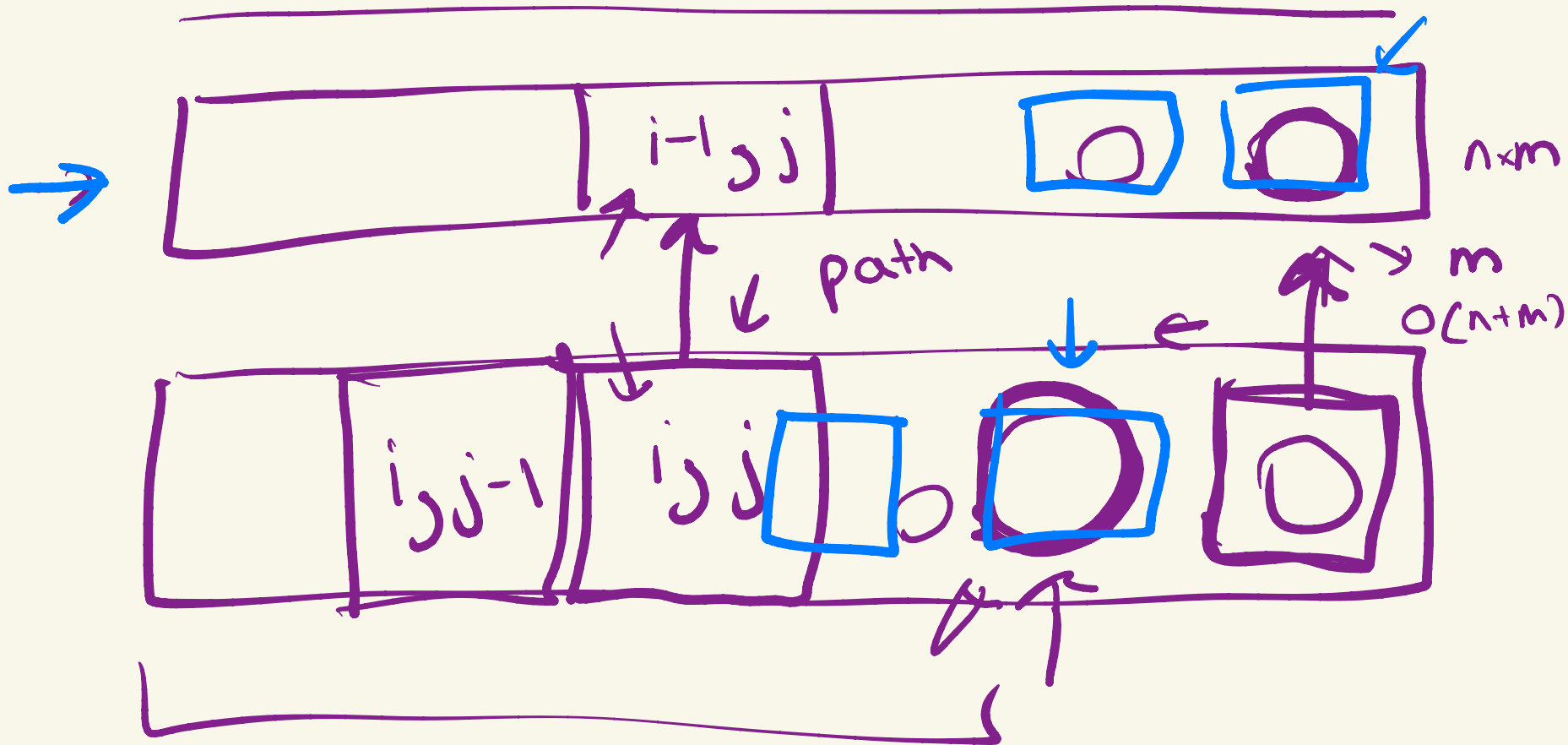
*

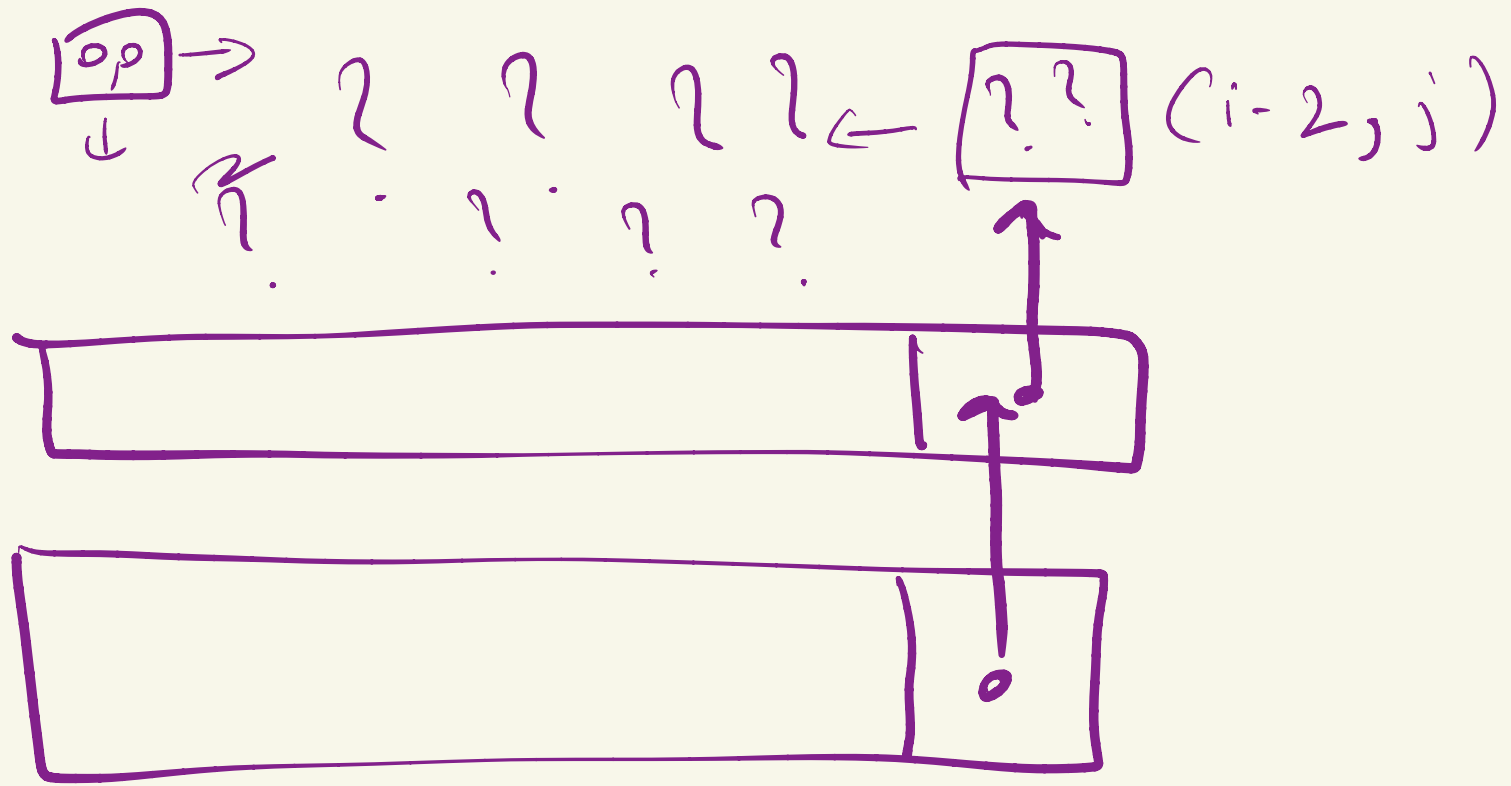
Note for Answer Construction



- We cannot apply space optimization when we want to construct the answer, as we require all the states along with the best choice for every state.









Problem 1

LCS:

https://atcoder.jp/contests/dp/tasks/dp_f

Longest Common Subsequence

$s = \underline{a} \underline{x} y \underline{b}$

axb

$t = \underline{a} b y \underline{a} \underline{b}$

ayb

a x y b

a b y x b

azb ↙ LCS

Answer constr.

Longest common subsequence

→ dp problem → LCS length

$\downarrow \downarrow$
a x y b
└─┬─┬─┬─┐
└─┬─┬─┬─┐
└─┬─┬─┬─┐
a b y x b

i = 0

j = 0

1 +

* imp

$$dp_{i,j} = \begin{cases} s_i = t_j & 1 + dp_{i+1,j+1} \end{cases}$$

$$s_i \neq t_j$$

s_i

t_j

$dp_{i,j+1}$

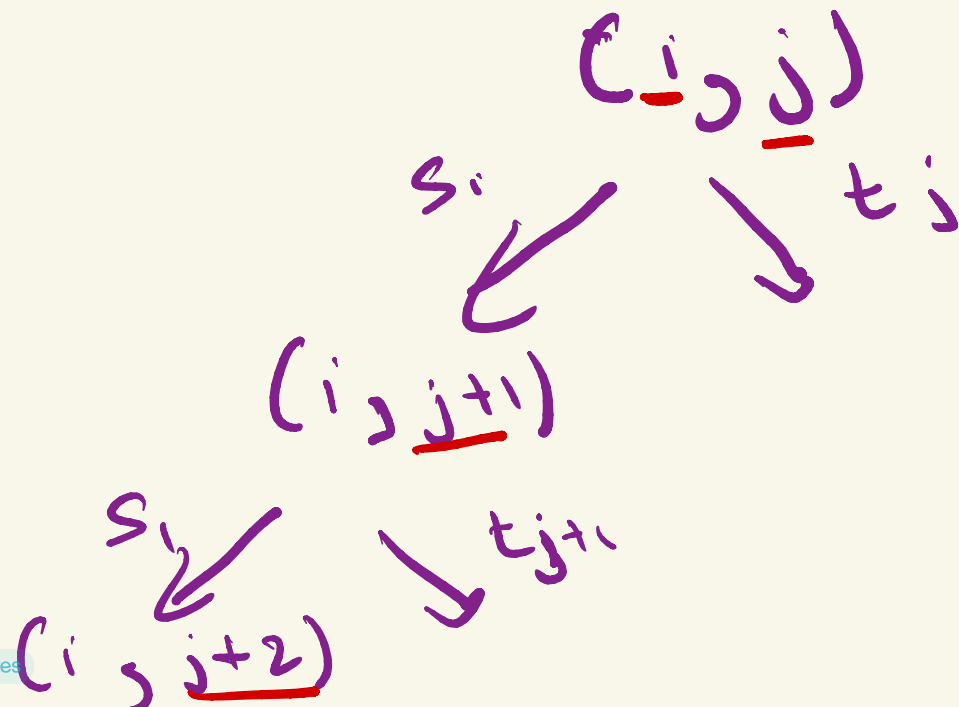
$dp_{i+1,j}$

$dp_{i+1,j+1}$

not required

$\begin{bmatrix} \times & dp_{i+1,j+1} \end{bmatrix}$

(i, j) $\rightarrow (i, j+1) \Rightarrow (i, j+2)$
↓



Recursively
all cases
are being
handled

Iterative dp

$dp_{i,j} \Rightarrow$ LCS length
 $\underbrace{s_0 \dots i}_{\text{string 1}} \text{ to } \underbrace{j \dots j}_{\text{string 2}}$

i, j

$$\begin{aligned} & \rightarrow s_i = t_j \\ & \quad \underline{1 + dp_{i+1, j-1}} \end{aligned}$$

$$\rightarrow \underline{s_i \neq t_j} ?$$

$$\begin{aligned} \text{fixing } t_j & \rightarrow dp_{i-1, j} \\ \text{fixing } s_i & \rightarrow dp_{i, j-1} \end{aligned} \quad \left. \vphantom{\begin{aligned} \text{fixing } t_j & \rightarrow dp_{i-1, j} \\ \text{fixing } s_i & \rightarrow dp_{i, j-1} \end{aligned}} \right] \underline{\underline{\text{max}}}$$

Recursive DP

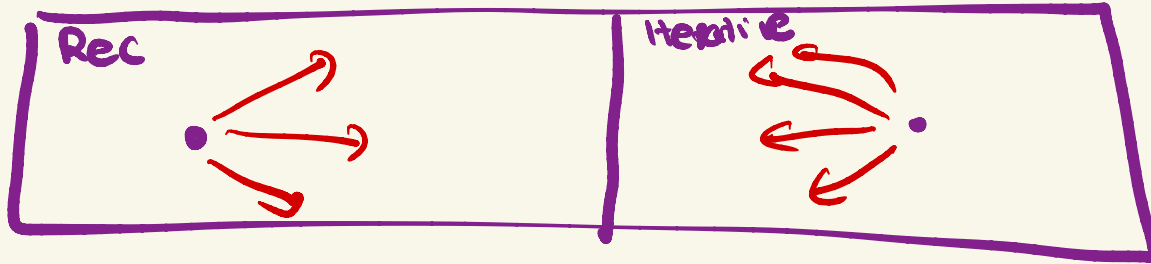
→ $dp[0][0]$

Iterative DP

→ $dp[n-1][m-1]$
0 based

$dp[n][m]$

1 based





Problem 1 Solution

DP sol??

```
for(int i = 1; i <= s.size(); i++)  
{  
    for(int j = 1; j <= t.size(); j++)  
    {  
        if(s[i - 1] == t[j - 1]) dp[i][j] = 1 + dp[i - 1][j - 1];  
        else dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);  
    }  
}
```

```
int x = s.size(), y = t.size();
```

```
string answer = "";
```

```
while(x > 0 && y > 0)
```

```
{  
    if (s[x-1] == t[y-1])  
    {  
        if(dp[x][y] > dp[x-1][y] && dp[x][y] > dp[x][y-1]) answer += s[x-1], x--, y--;  
        else if(dp[x-1][y] == dp[x][y]) x--;  
        else if(dp[x][y-1] == dp[x][y]) y--;  
    }  
}
```

[print(reverse(answer))]

$dp[n+1][m+1]$

$dp[n][m]$

a
b
x
y
b

	a	b	y	b
a	0	0	0	0
b	0	1	1	1
x	0	1	2	2
y	0	1	2	2
b	0	1	1	3
	0	1	1	3

abyb

b y b a

a b y b



Non-integer DP Parameters

- How will you store the dp states if instead of integer parameters you had a string or a vector or a map or any complex data type?
- Use a map instead of an array.
- Trade off b/w `unordered map` and `normal (tree) map`
- Trade off `map<pair<int, string>> DP` or `vector<map<string>> DP`

dp $i, j, k \dots$ = something
 ↑ ↑ ↑
 is an integer ??
 ↓ string, set, vector

$dp_{ij} \rightarrow \text{something}$
 $\begin{matrix} \text{int} & & \text{string} \\ \nearrow & \nearrow & \\ (0 \dots n) & m & n \times m \end{matrix}$
 Store ??

$\text{map} \langle \text{pair} \langle \text{int}, \text{string} \rangle, \text{int} \rangle dp$

$TC \xrightarrow{\text{BAD}} \log(n \times m) \quad dp[\text{index}, \underline{s}]$

Time complexity?

$$O(\log_2(n \times m))$$

$$O(\log_2(m))$$

dp: i j
↑ ↑
(0..n) (0..m)
 $n \leq 2^{10}$

vector<map<string, int>> dp(n);

map dp[i][j] ← key for that map

$$dp[i][j][k] \rightarrow \sum_{l=i}^{j-1} dp[l][j][k]$$

vector (n)

$\mapsto$ map $\{j, k\}$

Problem [From Google Kickstart]

Priyansh

Global Rank 6

Palindrome Free Strings: <https://zibada.guru/gcj/ks2022a/problems/#C>

dp[i][j] → index i, string j ^{string made till now}

↳ 1 → make a string with
no ≥ |s| palindromes

↳ 0

$$dp[i][j] = \begin{cases} \text{j itself has a } \geq 5 \text{ len} \\ \text{palindrome} & \text{False/0} \\ \hline s_i = 0/1 & dp(i+1, j+s_i) \\ \hline s_i = ? & \begin{cases} dp(i+1, j+'0') \rightarrow T/F \\ \text{OR } dp(i+1, j+'1') \rightarrow T/F \end{cases} \end{cases}$$

$dp_{ij} \rightarrow i$ $j \rightarrow \text{string}$
 $\hookrightarrow 2 \times 10^4$ $\hookrightarrow 2^{(2 \times 10^4)}$??

?? ? ? ?

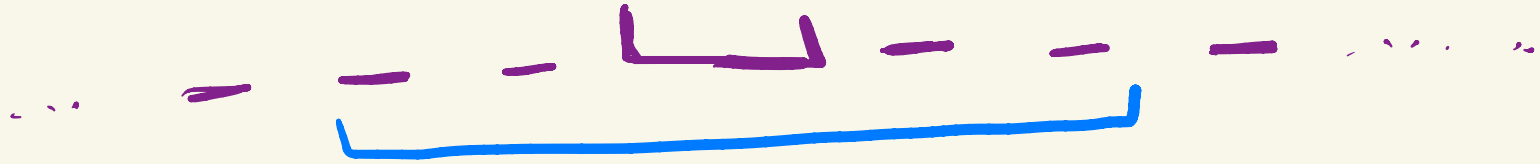
Optimise

odd length

len ≥ 5

5, 7, 9, ...

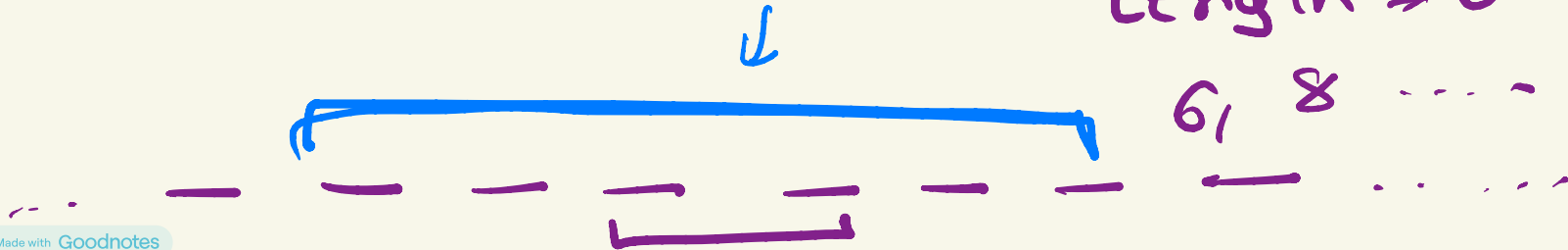
* Rad



even length

Length ≥ 6

6, 8, ...



$dp[i][j]$ $\Rightarrow$ ~~the~~ entire string
 $\hookrightarrow$ last 6 characters
 placed

$$2^6 = \underline{64} \times 6$$

$$n \times 64 \times 6$$

```
vector < map < string, int > > dp(n);
```

dp structure

→ lot of useless states

Recursive > iterative



S & 6
len
palindrome

```
auto f = [&](int index, string s, auto &&F) -> int {  
    s = shorten(s); ← last 6 characters  
    → if(checkPalindrome(s)) return false;  
    if(index == n) return true; ← ^  
    if(dp[index].count(s)) return dp[index][s];  
    ← if(S[index] != '?') return dp[index][s] = F(index + 1, s + S[index], F);  
    → return dp[index][s] = F(index + 1, s + "1", F) || F(index + 1, s + "0", F);  
};  
  
→ if(f(0, "", f)) cout << "Possible\n";  
else cout << "Impossible\n";
```

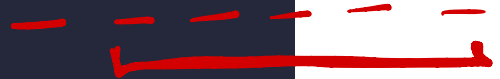


```
auto shorten = [&](string s) -> string {  
    → if(s.size() <= 6) return s;  
    return string(s.end() - 6, s.end());  
};
```

returning last
6 characters

```
auto checkPalindrome = [&](string s) -> bool {  
    → if(s.size() < 5) return false;  
    → bool palindrome = true;  
    for(int i = 0; i < s.size(); i++) {  
        palindrome &= (s[i] == s[s.size() - 1 - i]);  
    }  
    if(s.size() == 6) {  
        bool check = true; S  
        for(int i = 1; i < s.size(); i++) {  
            check &= (s[i] == s[s.size() - 1 - (i - 1)]);  
        }  
        if(check) return true;  
    }  
    return palindrome;  
};
```

length 5 6



Problem Solving

Homework Problem

Minimising the sum: <https://codeforces.com/problemset/problem/1969/C>

Problem Solving Solution

```
vector dp(k + 1, vector(n, -1));

function<int(int, int)>f = [&](int index, int opsLeft) {
    if(index == n) return 0;
    if(dp[opsLeft][index] != -1) return dp[opsLeft][index];

    int answer = a[index] + f(index + 1, opsLeft);
    int mn = a[index];

    for(int i = 1; i <= opsLeft; i++) {
        if(index + i == n) break;
        mn = min(mn, a[index + i]);
        answer = min(answer, (i + 1) * mn + f(index + i + 1, opsLeft - i));
    }

    return dp[opsLeft][index] = answer;
};
```